A 3.4.4 Distributed databases (HL Only)

A 3.4.4 Describe the features of distributed databases (HL Only)

Explore distributed databases. Learn key features: concurrency control, consistency, partitioning, security, transparency, fault tolerance, global queries, replication, scalability. Understand ACID for reliable transactions and the crucial need for data consistency across multiple locations.

Introduction to Distributed Databases

In today's data-driven world, the sheer volume and velocity of information often exceed the capabilities of traditional, centralised database systems. Imagine the vast amounts of user data handled by social media platforms, the real-time transactional data of global e-commerce sites, or the sensor readings from a worldwide network of Internet of Things (IoT) devices. Managing such massive datasets efficiently and reliably necessitates a different approach – the distributed database.

What is a Distributed Database?

A distributed database is a database in which data is stored across multiple physical locations, such as different servers or nodes within a network. These nodes can be in the same geographical area or spread across continents. Unlike a centralised database where all data resides on a single server, a distributed database partitions and replicates data across these interconnected sites.

Why Use a Distributed Database?

The primary motivations for adopting a distributed database architecture include:

- **Scalability:** Handling increasing data volumes and user traffic by adding more nodes to the system.
- **Availability:** Ensuring continuous operation even if some nodes fail, as data is often replicated across multiple locations.
- **Performance:** Improving query response times by distributing the workload and potentially locating data closer to users.
- **Fault Tolerance:** Enhancing the system's resilience to failures by allowing other nodes to take over if one goes offline.

These advantages directly lead to the key features we will explore in detail throughout this ebook.

The Cornerstones - Data Consistency and ACID Properties

When data is spread across multiple locations, ensuring its accuracy and uniformity becomes a significant challenge. Two fundamental concepts are crucial in addressing this: data consistency and ACID properties.

The Need to Maintain Data Consistency in a Distributed Database

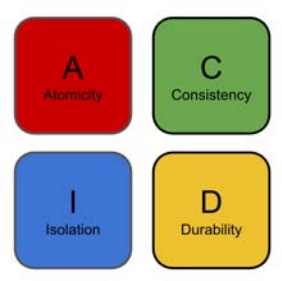
Data consistency in a distributed database refers to the requirement that all copies of the same data, residing on different nodes, should reflect the same value at any given point in time. Imagine an online banking system storing your account balance on several servers. If you make a withdrawal, this change must be reflected consistently across all copies of your account information.

Without proper consistency mechanisms, a distributed database could suffer from various issues:

- **Lost Updates:** Changes made to one copy of the data might be overwritten by outdated information from another copy.
- **Dirty Reads:** A transaction might read data that has been modified by another transaction but not yet committed, leading to incorrect results.
- **Non-Repeatable Reads:** Reading the same data within a single transaction might yield different results because another transaction has modified it in the interim.

Maintaining data consistency in a distributed environment is inherently complex due to network latency, potential node failures, and the need for concurrent access from multiple users. Various consistency models exist, offering different trade-offs between consistency and performance.

The Role of ACID to Ensure Reliable Processing of Transactions in Distributed Databases



Transactions are sequences of operations that are treated as a single logical unit of work. In a database system, ACID is a set of properties that guarantee database transactions are processed reliably. These properties are even more critical in distributed databases to maintain data integrity across multiple nodes.

Let's examine each ACID property in the context of distributed databases:

- **Atomicity:** A transaction must be treated as an indivisible unit; all operations are successfully completed (committed), or none are (rolled back). In a distributed system, this means that if a transaction involves updates across multiple nodes, either all these updates must succeed, or the system must ensure that all partial updates are undone, leaving the database in a consistent state. This often requires sophisticated coordination protocols across the distributed nodes.
- **Consistency:** A transaction must move the database from one valid state to another. All data written to the database must be valid according to defined rules, including constraints, cascades, and triggers. In a distributed database, ensuring consistency requires that even with concurrent transactions and potential failures, the overall state of the distributed data remains valid.
- **Isolation:** Concurrent transactions should execute in a way that makes it appear as if they are running sequentially. The intermediate state of one transaction should not be visible to other concurrent transactions. In a distributed database, achieving isolation can be challenging due to the distributed nature of the data and the potential for concurrent access from different locations. Mechanisms like distributed locking are used to prevent transactions from interfering with each other.
- **Durability:** Once a transaction is committed, the changes made to the database are permanent and will survive even system failures (e.g., power outages, node crashes). In a distributed database, durability is often achieved through replication, where committed data is stored on multiple nodes. If one node fails, the other replicas can recover the data.

Implementing and guaranteeing ACID properties in a distributed database is significantly more complex than in a centralised system. Distributed transaction management protocols are employed to coordinate transactions across multiple nodes and ensure these properties are upheld.

Key Features of Distributed Databases

Now, let's delve into the specific features that define distributed database systems:

- **Concurrency Control:** In a distributed database, multiple users and applications may try to access and modify data concurrently across different nodes. Concurrency control mechanisms are essential to manage these simultaneous operations and prevent conflicts, ensuring data

consistency and isolation. Techniques like distributed locking (where a lock on a data item might span across multiple nodes) and timestamping (assigning timestamps to transactions to manage their order) are employed to achieve this.

- **Data Consistency:** As discussed earlier, this feature ensures that all copies of data across the distributed nodes remain synchronised. Various consistency models exist, ranging from strong consistency (where all replicas are updated before the transaction commits, offering a single system image) to eventual consistency (where updates might take some time to propagate to all replicas, but eventually, all copies will be consistent). The choice of consistency model depends on the application's data accuracy and availability requirements. ACID properties play a crucial role in achieving strong consistency within transactions.
- **Data Partitioning:** To improve performance and scalability, distributed databases often divide the data into smaller, independent units called partitions or shards. These partitions are then distributed across different nodes. There are two main types of partitioning:
    - **Horizontal Partitioning (Sharding):** Rows of a table are divided into multiple tables based on a certain key or range of keys. For example, customer data might be partitioned based on geographical region.
    - **Vertical Partitioning:** Columns of a table are divided into multiple tables. This might be done if certain columns are accessed more frequently than others. Partitioning allows for parallel processing of queries and reduces the amount of data that each node needs to manage. However, it also introduces challenges in managing distributed queries that span across multiple partitions and maintaining referential integrity.
- **Data Security:** Securing a distributed database requires addressing the vulnerabilities inherent in a networked environment. This includes securing each individual node, the communication channels between them (using encryption protocols like TLS), and implementing robust authentication and authorisation mechanisms to control access to data across the distributed system. Measures to prevent network security attacks, such as those affecting healthcare institutions, become critical.
- **Distribution Transparency:** An ideal distributed database provides distribution transparency, meaning that the users and applications interacting with the database are unaware that the data is distributed across multiple locations. This transparency can occur at different levels:
    - **Location Transparency:** Users do not need to know the physical location of the data they are accessing. The system handles routing the queries to the correct nodes.
    - **Access Transparency:** Users interact with the distributed database using the same query language (e.g., SQL) as they would with a centralised database, without knowing how the data is partitioned or replicated.
    - **Failure Transparency:** The system can continue to operate even if some nodes fail, without the user noticing any interruption in service. This is closely related to fault tolerance.
- **Fault Tolerance:** Distributed databases are designed to be more resilient to failures than centralised systems. If one or more nodes fail, the system can continue to operate, often with minimal or no data loss, due to data replication. Techniques like automatic failover (where another replica automatically takes over if a primary node fails) contribute to fault tolerance.

- **Global Query Processing:** When a user submits a query to a distributed database, the system needs to determine the optimal way to execute that query across the distributed data. This involves breaking down the query into sub-queries, identifying which nodes contain the relevant data partitions, and coordinating the execution of these sub-queries. Global query processing aims to minimise data transfer across the network and optimise the overall query execution time.
- **Location Transparency:** As mentioned under distribution transparency, location transparency allows users to access data without knowing its physical location. The distributed database management system (DDBMS) handles the task of locating and retrieving the requested data from the appropriate nodes.
- **Replication:** Replication involves creating and maintaining multiple copies (replicas) of data on different nodes within the distributed database. This is a key technique for improving data availability and fault tolerance. If one node containing a replica fails, other replicas can still access data. Replication can be synchronous (where all replicas are updated before a transaction commits, ensuring strong consistency) or asynchronous (where updates might be applied to replicas after the primary update, potentially leading to eventual consistency).
- **Scalability:** Distributed databases are inherently designed to be scalable. **Horizontal scalability** is achieved by adding more nodes to the system to distribute the workload and storage capacity. **Vertical scalability**, which involves increasing individual nodes' resources (e.g., CPU, memory), is often more limited and expensive in distributed systems. Horizontal scalability allows distributed databases to handle massive data growth and high transaction rates.

Conclusion

Distributed databases are a fundamental technology for managing the ever-increasing demands of modern data-intensive applications. Understanding their core features, including the critical need for data consistency ensured by mechanisms like ACID properties, and the benefits of concurrency control, partitioning, security, transparency, fault tolerance, global query processing, replication, and scalability, is essential for any aspiring computer scientist.

As you continue your journey in IB Computer Science, remember that the design and implementation of distributed databases involve complex trade-offs. The choice of specific features and techniques depends heavily on the application's specific requirements, including factors like the need for strong consistency versus high availability, the expected data volume and transaction rate, and the tolerance for potential failures.

This section has provided a foundational understanding of distributed databases. Further exploration into specific distributed database systems, consistency models, transaction

management protocols, and query optimisation techniques will deepen your knowledge in this exciting and crucial area of computer science.